

EHMbrAI

AI-based versus traditional Engine Health Monitoring:
a gas path analysis testbed on the CFM56-7B with synthetic ground
truth

— (anonymous in the public repository) —

July 3, 2026

Living document: updated at every project milestone.

Contents

Notation and acronyms	3
1 Introduction	5
1.1 Motivation	5
1.2 Objective	6
1.3 Hypotheses	6
1.4 Contributions	6
1.5 Document structure	7
2 Background	9
2.1 How a two-spool turbofan works	9
2.2 The subject engine: CFM56-7B26	10
2.3 What is measured, and why parameters must be corrected . .	11
2.4 Gas Path Analysis	11
2.4.1 Health parameters	11
2.4.2 The linear model	11
2.4.3 Estimation, and why it is hard	12
2.4.4 Smearing	12
2.5 The physics of degradation	13
3 Methodology and testbed architecture	14
3.1 Overall architecture	14
3.2 The machine-learning toolbox, briefly	15
3.3 Methodological safeguards	16
3.4 Reproducibility and infrastructure	16
3.5 Software architecture and computational pipeline	17
3.5.1 Library modules (<code>src/ehmbrain/</code>)	17
3.5.2 Executable scripts (<code>scripts/</code>)	18
3.5.3 Test suite (<code>tests/</code>)	19
3.6 Comparison metrics	19
4 Thermodynamic twin of the CFM56-7B26	21
4.1 What a cycle model is	21
4.2 Strategy: adapt, don't build from scratch	22

4.3	Design point (WP1.1)	22
4.4	Calibration against measured data (WP1.2)	24
4.4.1	Data sources and philosophy	24
4.4.2	Calibration iterations	24
4.4.3	Idle convergence: diagnosis and cure	26
4.5	Baseline decks (WP1.3)	26
4.5.1	Corrected-parameter exponents and the corrected-space baseline	27
4.6	Influence coefficient matrix and observability (WP1.4)	28
4.6.1	Health-parameter injection	28
4.6.2	The ICM at the reference conditions	29
4.6.3	Observability: the quantitative case for hypothesis H2	29
4.6.4	Thermodynamics cross-check	30
4.7	Declared limitations	31
4.8	Automated verification	31
5	Work in progress and planning	32
5.1	Phase F1: closed	32
5.2	Remaining phases	32
5.3	Milestone log	32
	References	34

Notation and acronyms

Acronyms

BPR	bypass ratio	LOO	leave-one-out (cross-validation)
CEA	NASA Chemical Equilibrium with Applications	LPC	low-pressure compressor (booster)
CI	continuous integration	LPT	low-pressure turbine
C-MAPSS	NASA turbofan simulation benchmark	MAE	mean absolute error
EEDB	ICAO Engine Emissions Data-bank	OPR	overall pressure ratio
EGT(M)	exhaust gas temperature (margin)	PC	power code: fraction of reference thrust
EHM	engine health monitoring	PCS	Physics-Consistency Score (ours, C5)
FADEC	full-authority digital engine control	PHM	prognostics and health management
FAR	fuel-air ratio	RMSE	root-mean-square error
FOD	foreign-object damage	RUL	remaining useful life
FPR	false-positive rate	SAC	single annular combustor
GPA	gas path analysis	SHAP	Shapley additive explanations
HPC	high-pressure compressor	SLS	sea-level static
HPT	high-pressure turbine	SVD	singular value decomposition
ICM	influence coefficient matrix	TCDS	type-certificate data sheet
ISA	international standard atmosphere	TSFC	thrust-specific fuel consumption
N1, N2	low-/high-pressure spool speeds	WLS	weighted least squares

Symbols

θ, δ	inlet total temperature / pressure ratios to ISA sea level	$\mathbf{H}$	influence coefficient matrix
$\Delta\eta$	component efficiency deviation [%]	$\mathbf{Q}, \mathbf{R}$	process / measurement noise covariance
$\Delta\Gamma$	component flow-capacity deviation [%]	$\mathbf{P}_0, \lambda$	prior shape / strength of the WLS regularization
$\mathbf{x}$	health-parameter vector (10 components)	$N1_c$	corrected fan speed $N1/\sqrt{\theta}$
$\Delta\mathbf{z}$	measurement deviations from baseline	T_{t4}	combustor-exit total temperature
W_f, F_n	fuel flow, net thrust	ΔT_{ISA} (dT _s)	ambient offset from the ISA day

Reading order. Every technical term is defined at first use; Chapter 2 builds the engine and GPA foundations, Section 3.2 the machine-learning toolbox. The hypotheses table (Table 1.1) can be read superficially on a first pass and revisited after those two stops.

Chapter 1

Introduction

1.1 Motivation

A commercial jet engine degrades from its very first flight. Dust and pollution stick to compressor blades, hot-section parts creep and oxidize, tip clearances open up, seals wear. The airline cannot see any of this directly —the engine stays on the wing for years— so it watches the engine’s *symptoms* instead: the temperatures, pressures, shaft speeds and fuel flow recorded on every flight. The discipline that turns those symptoms into maintenance decisions is called *Engine Health Monitoring* (EHM). The stakes are large: a single shop visit for an engine overhaul costs millions of dollars, and an unscheduled engine removal disrupts an entire fleet schedule.

The classical analytical core of EHM, in use since the 1970s, is *Gas Path Analysis* (GPA) [1]. The idea: each physical fault (a dirty compressor, an eroded turbine) leaves a characteristic fingerprint on the measured parameters; by inverting a thermodynamic model of the engine, one can estimate *which* component has deteriorated and *by how much* from the measurements alone. Around GPA the industry built a mature toolbox —baseline models, exponential trend smoothing, Kalman filters, expert fault-isolation rules [2]— which this project collectively calls *traditional EHM*.

Over the last decade, machine learning has produced striking results in the adjacent research field of prognostics: predicting the *Remaining Useful Life* (RUL) of an engine, i.e. how many flights remain before it must come off the wing, mostly on synthetic benchmark datasets such as NASA’s C-MAPSS [3] and N-CMAPSS [4]. Yet the literature has a structural weakness: **AI methods are almost never compared against a traditional pipeline implemented with equal care**, on the same data, with the same metrics and the same tuning effort. The result is a stream of comparisons against straw men, which makes it impossible to know what AI actually contributes, under which conditions, and at what cost.

1.2 Objective

This project builds a reproducible testbed in which both EHM families—traditional and AI-based—observe *exactly the same data* from a synthetic fleet of CFM56-7B26 turbofans and are scored with *exactly the same metrics*. The methodological key is that the synthetic data carries **complete ground truth**: the true health state of every engine component (its efficiency and flow-capacity deviations, defined in Section 2.4) at every flight. With real airline data this is impossible—nobody knows the true internal state of an engine on the wing—but with synthetic data one can measure not only whether a method detects or predicts well, but whether it *attributes the damage to the correct component*. That attribution question is precisely where traditional GPA is weakest (the *smearing* phenomenon, Section 2.4) and where AI is least transparent (the black-box problem).

Since neither a performance model of the engine nor degradation data were available to the project, both are built from scratch: a thermodynamic “digital twin” of the CFM56-7B26 in pyCycle [5], calibrated against certified public data (Chapter 4), and a fleet degradation generator with physically derived fault signatures (planned, Chapter 5).

1.3 Hypotheses

Each hypothesis is formalized with quantitative thresholds *before* any test-set result is seen (the pre-registration protocol of Section 3.3). All are tested with a Wilcoxon signed-rank test paired by engine, Holm correction for multiple comparisons ($\alpha = 0.05$), and BCa bootstrap confidence intervals. In plain words: for every comparison we check that the improvement is statistically significant, not a fluke of one lucky engine. The table below necessarily uses terms defined later—the GPA quantities in Chapter 2, the machine-learning methods in Section 3.2, every metric in the metrics section of Chapter 3, and all acronyms in the notation chapter—so it is meant to be skimmed now and revisited after those stops.

Refuting any hypothesis is itself a publishable result: pre-registration is what makes the refutation credible. Reformulating hypotheses after the repository tag `prereg-v1` is forbidden.

1.4 Contributions

- C1. SynCFM56:** the first public synthetic GPA dataset for a specific commercial engine (CFM56-7B26), with per-flight health-parameter ground truth and an ACARS-style snapshot format (one takeoff report and one cruise report per flight, the way real airline EHM data actually arrives), more realistic than the continuous time series of C-MAPSS.

Table 1.1: Project hypotheses and their confirmation criteria.

ID	Hypothesis	Confirmation criterion
H1	AI detects faults earlier and with fewer false alarms than trend monitoring	median lead time $\geq +20\%$ and false-positive rate $\leq 0.5\times$ versus trending, at a fixed recall of 0.9; $p < 0.05$
H2	AI isolates faults with overlapping signatures better	+10 percentage points of isolation accuracy on the “confusable” subset (fault signatures less than 15° apart in the influence-matrix space) versus weighted-least-squares GPA; $p < 0.05$
H3	AI predicts RUL better	simultaneous improvement in RMSE and in the asymmetric NASA PHM08 score; $p < 0.05$
H4	The physics-informed hybrid dominates when data is scarce	RUL RMSE $\leq$ pure AI with 100 % of the training data and strictly better with 10 % and 25 %; $p < 0.05$
H5	Conformal intervals are better calibrated than the Kalman covariance	smaller $ \text{empirical coverage} - 0.9 $ with mean interval width no worse than +20 %

- C2.** A head-to-head comparison under a **pre-registered protocol** with a symmetric tuning budget (the same number of Optuna trials) for both families.
- C3.** A physics-informed hybrid with three separately ablated mechanisms: residual features against the digital twin, physically constrained loss functions, and GPA $\rightarrow$ ML stacking.
- C4.** Compared uncertainty quantification: conformal-prediction RUL intervals versus the Kalman filter covariance.
- C5.** The **Physics-Consistency Score (PCS)**: a new explainability metric that projects SHAP attributions into the physical health-parameter space through the influence coefficient matrix.
- C6.** A determinability map: a sensors $\times$ noise $\times$ data-size ablation showing *when* each approach wins.
- C7.** Sim-to-real validation: the full methodology is re-run on N-CMAPSS to check that the method ranking is not an artifact of our own simulator.

1.5 Document structure

Chapter 2 builds the technical foundations assuming no prior knowledge of jet engines or GPA: how a turbofan works, what is measured on the wing, and the mathematics of gas path diagnosis. Chapter 3 describes the testbed architecture and the methodological safeguards. Chapter 4 documents the thermodynamic twin of the CFM56-7B26 and its calibration against certified

data —the work completed to date— with quantitative evidence of every step. Chapter 5 summarizes the remaining phases and their acceptance criteria.

Chapter 2

Background

This chapter assumes no prior knowledge of gas turbines. It introduces the engine, the measurements available in service, the mathematics of gas path diagnosis, and the physics of degradation —everything needed to follow the rest of the document.

2.1 How a two-spool turbofan works

A turbofan produces thrust by accelerating air backwards. Air enters through the *fan* (the large visible rotor); most of it —the *bypass* stream— goes around the core and exits through the bypass nozzle, producing the majority of the thrust. The rest enters the *core*: it is compressed by a low-pressure compressor (the *booster*) and a high-pressure compressor (HPC), heated by burning fuel in the *combustor*, and expanded through a high-pressure turbine (HPT) and a low-pressure turbine (LPT) that extract the work needed to drive the compressors, before leaving through the core nozzle.

The machine is arranged in two mechanically independent *spools* (concentric shafts):

- the **low-pressure (LP) spool**: fan + booster driven by the LPT, rotating at speed **N1**;
- the **high-pressure (HP) spool**: HPC driven by the HPT, rotating at speed **N2**.

The ratio of bypass to core mass flow is the *bypass ratio* (BPR); the ratio of compressor exit pressure to inlet pressure is the *overall pressure ratio* (OPR). Higher BPR and OPR generally mean better fuel efficiency, measured as *thrust-specific fuel consumption* (TSFC): fuel flow per unit of thrust.

2.2 The subject engine: CFM56-7B26

The CFM56-7B (CFM International, a Safran–GE joint venture) powers the Boeing 737NG family and is one of the most widely operated commercial engines in history, which guarantees abundant public type data. The **-7B26** variant (117 kN takeoff thrust, Boeing 737-800) is chosen as the most common one. Its type-certificate data sheet (TCDS EASA E.004, issue 07 [6]) describes: a single-stage fan, 3-stage booster, 9-stage HPC, annular combustor, single-stage HPT and 4-stage LPT, governed by a dual-channel FADEC (a full-authority digital engine control—the computer that meters fuel). The pilot-facing thrust-setting parameter is N1, not engine pressure ratio.

Table 2.1: Certified CFM56-7B26 data used as the calibration contract. Sources: TCDS EASA E.004 issue 07 [6] and the ICAO Engine Emissions Databank, edition 03/2026, UID 3CM033 [7].

Quantity	Value	Source	Status
Takeoff thrust	11 699 daN (= 116.99 kN)	TCDS	verified
Maximum continuous thrust	11 521 daN	TCDS	verified
N1 redline (104 %)	5382 rpm $\Rightarrow$ 100 % = 5175 rpm	TCDS	verified
N2 redline (105 %)	15 183 rpm $\Rightarrow$ 100 % = 14 460 rpm	TCDS	verified
Max. EGT, takeoff / continuous	950 °C / 925 °C (displayed)	TCDS	verified
Dry weight	2386 kg	TCDS	verified
Bypass ratio	5.1	EEDB	verified
Pressure ratio (sea-level static, rated)	27.61	EEDB	verified
Fuel flow at 100/85/30/7 % F_{00}	1.221 / 0.999 / 0.338 / 0.113 kg s ⁻¹	EEDB	verified

Two subtleties from the TCDS matter for modeling and are frequently overlooked:

- The certified *exhaust gas temperature* (EGT)—the single most important health indicator, because it rises as the engine deteriorates—is measured at station **T49.5** (the stage-2 LPT nozzle), *not* at the HPT exit. Moreover, the *displayed* cockpit value adds a “shunt” function of +30 °C (for single-annular-combustor engines, active above 8500 rpm N2) plus a model-dependent “trim” (zero for the -7B26).
- Air bleed limits (air extracted from the compressor for the aircraft’s cabin and anti-ice systems) are tabulated per extraction stage and shaft speed; at rated speed the 9th-stage bleed is capped at 7 % of core flow.

2.3 What is measured, and why parameters must be corrected

Engine locations are numbered per the SAE ARP755A standard [8]: station 2 is the fan inlet, 21 the fan exit, 25 the HPC inlet, 3 the HPC exit, 4 the combustor exit, 45 the HPT exit, and 5 the LPT exit. The measurements available in airline service (the “cockpit set”) are N1, N2, EGT and fuel flow W_f ; some installations add pressures and temperatures at stations 25 and 3.

A raw measurement is meaningless on its own because it depends on the day: the same healthy engine reads a higher EGT on a hot day at a high-altitude airport than on a cold day at sea level. To compare flights, measurements are *corrected* to standard sea-level conditions using $\theta = T_{t2}/288.15 \text{ K}$ and $\delta = P_{t2}/101.325 \text{ kPa}$ (the ratios of inlet total temperature and pressure to their standard-day values):

$$N_{\text{corr}} = \frac{N}{\sqrt{\theta}}, \quad W_{f,\text{corr}} = \frac{W_f}{\delta \theta^a}, \quad \text{EGT}_{\text{corr}} = \frac{\text{EGT}}{\theta}. \quad (2.1)$$

The exponent a (typically 0.5–0.7) is often copied from generic tables; here it will instead be fitted by regression on sweeps of our own thermodynamic model, removing a classical source of systematic baseline error.

2.4 Gas Path Analysis

2.4.1 Health parameters

GPA describes the internal state of the engine with two numbers per turbomachine: the deviation of its *isentropic efficiency* $\Delta\eta$ (how much of the ideal work it actually achieves—dirt and wear reduce it) and the deviation of its *flow capacity* $\Delta\Gamma$ (how much corrected flow it swallows at a given speed—fouling reduces it in compressors; turbine erosion, which opens the effective nozzle area, increases it). With five turbomachines (fan, booster, HPC, HPT, LPT) the health state is the 10-component vector

$$\mathbf{x} = [\Delta\eta_{\text{fan}}, \Delta\Gamma_{\text{fan}}, \Delta\eta_{\text{LPC}}, \Delta\Gamma_{\text{LPC}}, \Delta\eta_{\text{HPC}}, \Delta\Gamma_{\text{HPC}}, \Delta\eta_{\text{HPT}}, \Delta\Gamma_{\text{HPT}}, \Delta\eta_{\text{LPT}}, \Delta\Gamma_{\text{LPT}}]^T, \quad (2.2)$$

expressed in percent relative to a new engine. $\mathbf{x}$ is what maintenance actually wants to know, and it is not directly measurable on the wing.

2.4.2 The linear model

What *is* measurable are deviations $\Delta\mathbf{z}$ of the corrected parameters from a *baseline* (the value a new engine would show at the same operating condition $\mathbf{u}$: altitude, Mach number, N1 setting, ambient temperature deviation,

bleed status). For the small deviations typical of in-service deterioration, measurement and health deviations are related linearly:

$$\Delta \mathbf{z} = \mathbf{H}(\mathbf{u}) \mathbf{x} + \mathbf{S} \mathbf{b} + \mathbf{v}, \quad \mathbf{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{R}), \quad (2.3)$$

where $\mathbf{H}$ is the *influence coefficient matrix* (ICM) —the Jacobian $\partial \mathbf{z} / \partial \mathbf{x}$, i.e. a table of “if the HPT loses 1 % efficiency, EGT rises by this much and fuel flow by that much”— computed by finite differences on the thermodynamic model at the operating condition $\mathbf{u}$; $\mathbf{b}$ collects sensor biases; and $\mathbf{v}$ is measurement noise with covariance $\mathbf{R}$.

2.4.3 Estimation, and why it is hard

Inverting Eq. (2.3) is an ill-posed problem. With 4 cockpit measurements and 10 unknowns the system is underdetermined ($\text{rank}(\mathbf{H}) \leq 4$): infinitely many health states explain the same measurements. The weighted-least-squares (WLS) answer regularizes the inversion with prior knowledge of plausible deterioration:

$$\hat{\mathbf{x}} = \left(\mathbf{H}^\top \mathbf{R}^{-1} \mathbf{H} + \lambda \mathbf{P}_0^{-1} \right)^{-1} \mathbf{H}^\top \mathbf{R}^{-1} \Delta \mathbf{z}, \quad (2.4)$$

where the two regularization pieces play distinct roles: $\mathbf{P}_0$ (a diagonal matrix of expected deterioration magnitudes squared) sets the *shape* of the prior — which parameters are allowed to move more — while the scalar λ sets its *strength*, trading fidelity to the measurements against fidelity to the prior. λ is a tuning knob of the traditional pipeline and receives the same optimization budget as the AI’s hyperparameters (Section 3.3).

Because deterioration evolves slowly over thousands of flights, tracking it as a *random walk* with a Kalman filter improves on flight-by-flight estimation. The Kalman filter is a recursive estimator that blends the previous health estimate with each new measurement, weighting each by its uncertainty:

$$\mathbf{x}_k = \mathbf{x}_{k-1} + \mathbf{w}_k, \quad \mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}); \quad \mathbf{z}_k = \mathbf{H}(\mathbf{u}_k) \mathbf{x}_k + \mathbf{v}_k, \quad (2.5)$$

with process noise $\mathbf{Q}$ calibrated to realistic degradation rates and the state optionally augmented with sensor biases [2]. At recorded compressor-wash events the recoverable part of the state is reset.

2.4.4 Smearing

When $\mathbf{H}$ is ill-conditioned —some fault signatures are nearly parallel— noise makes the estimator distribute a fault that is physically concentrated in one component across several: the *smearing* phenomenon, the classical weakness of linear GPA [9]. A singular-value decomposition (SVD) of the ICM quantifies this weakness (rank, condition number, angles between fault signatures) and defines the “confusable” subset used by hypothesis H2: fault pairs whose signatures are less than 15° apart.

2.5 The physics of degradation

Table 2.2: Degradation mechanisms modeled in this project and their typical effects, per the literature [10, 11].

Mechanism	Component	Typical effect	Time profile	Recoverable
Fouling (dirt buildup)	fan/LPC/HPC	$\Delta\Gamma$ $-1\ldots-4\%$; $\Delta\eta$ $-0.5\ldots-2.5\%$	saturating exponential	wash recovers 30–70 %
Erosion	compressors	$\Delta\eta$ $-0.5\ldots-1.5\%$	slow linear	no
Tip-clearance opening	HPC/HPT	$\Delta\eta$ $-0.5\ldots-1.5\%$	fast initially, then slow	no
Hot-section deterioration	HPT	$\Delta\eta$ $-1\ldots-3\%$; $\Delta\Gamma$ $+1\ldots+2\%$	linear/accelerating	no
Foreign-object damage (FOD)	fan/HPC	step $\Delta\eta$ $-0.5\ldots-2\%$	step (Poisson arrivals)	no
Sensor drift	any probe	e.g. EGT $+1\ldots+5\text{ }^\circ\text{C}$ per 1000 cycles	ramp	recalibration

The aggregate observable effect is the erosion of the *EGT margin*: the distance between the EGT reached on a hot-day takeoff and its certified limit. A new engine has 70–100 °C of margin; when the margin reaches zero the engine can no longer legally produce rated takeoff thrust on a hot day and must come off the wing. Its characteristic pattern over an engine’s life—fast early loss, slower steady erosion, sawtooth recoveries at each compressor wash—is what the fleet generator of phase F2 must reproduce. Table 2.2 summarizes the mechanisms, their signatures and their time profiles; its magnitudes parameterize that generator.

Chapter 3

Methodology and testbed architecture

3.1 Overall architecture

The testbed is organized in six chained phases, each closed by a milestone (*gate*) with a measurable acceptance criterion:

- F1. Thermodynamic twin** of the CFM56-7B26 in pyCycle: design point, off-design behavior, baseline decks over the flight envelope (pre-computed tables of what a healthy engine reads at every operating condition) and the influence coefficient matrix. Gate H1: error $\leq 3\%$ against certified data at the anchor points.
- F2. Synthetic fleet generator** (SynCFM56): per-mechanism health trajectories, a hierarchical fleet of ~ 100 engines run to failure, a sensor model (noise, bias, quantization, missing data) and ACARS-style snapshots with ground truth. Gate H2: dataset audited for realism and difficulty, no leakage between data splits, and a written datasheet.
- F3. Traditional EHM** reference: baselines and smoothed deviations, trend monitoring with persistence rules, WLS GPA (Eq. (2.4)), Kalman GPA (Eq. (2.5)), expert-rule fault isolation, and RUL by robust extrapolation of the EGT margin. Gate H3: full metrics on the test set.
- F4. AI-based EHM**: anomaly detection (autoencoders, Isolation Forest), multi-class fault diagnosis (gradient boosting, neural networks), RUL prognosis (LSTM/TCN/Transformer sequence models), the physics-informed hybrid, conformal uncertainty and physical explainability (PCS). Gate H4.
- F5. Pre-registered comparative evaluation**: common metrics, statistical tests, ablations, and sim-to-real validation on N-CMAPSS. Gate H5.

F6. Case studies, interactive dashboard, and final writing. Gate H6.

3.2 The machine-learning toolbox, briefly

For self-containedness, the AI techniques named above in one paragraph each:

Autoencoder (anomaly detection). A neural network trained to compress and then reconstruct *healthy* data. Fed a degraded snapshot, it reconstructs it poorly; the reconstruction error is the anomaly score. No fault labels needed.

Isolation Forest (anomaly detection). An ensemble of random trees; anomalous points are isolated in fewer random splits than normal ones. A classical non-neural baseline.

Gradient boosting / XGBoost (diagnosis). An ensemble of small decision trees, each correcting the errors of the previous ones. State of the art on tabular data; predicts the fault class of a snapshot window.

LSTM, TCN, Transformer (RUL prognosis). Three sequence-model families —recurrent, convolutional and attention-based— that consume a sliding window of past snapshots and regress the remaining useful life. Compared against each other in F4.

Conformal prediction (uncertainty). A distribution-free wrapper: from the model’s errors on a calibration set it builds prediction intervals with a mathematically guaranteed coverage rate (e.g. 90 %), regardless of the underlying model.

SHAP (explainability). Assigns each input feature its contribution to one specific prediction (a game-theoretic attribution). The PCS (contribution C5) projects these attributions through the ICM into health-parameter space to test their physical coherence.

Optuna (tuning). An automated hyperparameter search library; a “trial” is one training run with a candidate configuration. Both EHM families get the same trial budget.

Supporting infrastructure named later: *DVC* versions datasets the way git versions code; *MLflow* records every training run (configuration, metrics, artifacts) so experiments are traceable and repeatable.

3.3 Methodological safeguards

The credibility of an AI-versus-traditional comparison depends on eliminating the biases that plague the literature. Four safeguards are adopted:

Pre-registration. Hypotheses (Table 1.1), metrics, data splits (with file hashes), statistical tests and the stopping rule are frozen in the repository under a git tag (`prereg-v1`) *before* the test set is evaluated. This borrows a standard practice from medicine: you state what would convince you before looking at the answer, so you cannot move the goalposts afterwards.

Symmetric tuning budget. Every method has knobs. The traditional pipeline’s (alert thresholds, regularization λ , Kalman $\mathbf{Q}$, smoothing windows) and each AI family’s hyperparameters are tuned with the *same number* of automated search trials (Optuna), using only the training and validation engines.

Split by engine. The fleet is divided 70/10/20 engines into training, validation and test sets, and no engine ever appears in two sets. Splitting by flight instead of by engine—a common mistake—would let a model memorize an engine’s individual quirks and score unrealistically well. An automated leakage check runs in continuous integration.

A strong baseline. The traditional pipeline is implemented with the same care as the AI, every expert rule documented with its physical justification; smearing is *measured* against ground truth, not assumed.

3.4 Reproducibility and infrastructure

The entire project is a public repository¹ with a reproducible environment: Python 3.11 managed with `uv` and a fully pinned dependency lockfile; `pyCycle` 4.4 on OpenMDAO [12]; declarative YAML configuration; and automated tests (`pytest`) executed by continuous integration—a service that re-runs the whole test suite on every change, so a regression cannot land silently. Data and experiments of later phases will be versioned with DVC and MLflow respectively.

Three concrete decisions illustrate the intended standard of rigor:

- **A versioned calibration contract.** The model’s numerical targets (Table 2.1) live in a single file (`conf/cfm56_7b_targets.yaml`) carrying value, tolerance, source and status (`VERIFIED/TO_VERIFY`) per entry. Both the model runners and the tests read that file: changing a target is a visible version-control event, never a silent edit.

¹<https://github.com/cedarcreeks/EHMBraIn>

- **Dependency canary tests.** During environment setup, an incompatibility between pyCycle 4.4 and `numpy ≥ 2` was discovered (a failure inside the tabular thermodynamics package). The fix pins `numpy < 2`, and a smoke test that runs a complete engine cycle executes on every CI run, so any future dependency upgrade that breaks the solver fails loudly and immediately.
- **Auto-generated evidence.** Every result figure and table in this document is produced by a script (`scripts/make_report_assets.py`) that solves the model and writes the PDFs, the L^AT_EX table files (`paper/report/tables/*.tex`) and a provenance file (`assets.json`). No result number is ever copied by hand.
- **Compute-efficiency norms.** The repository carries binding engineering norms (`docs/engineering-norms.md`): long decomposable computations run in parallel across CPU cores (thermodynamic sweeps and ICM columns use one worker process per core — these are scalar Newton solves that gain nothing from a GPU), while the tensor workloads of phase F4 (network training, SHAP) run on the GPU (Apple-silicon MPS backend on the development machine).

3.5 Software architecture and computational pipeline

This section is the complete inventory of the project’s code: what each program does, how it does it, and what flows between them. It is updated whenever code lands.

3.5.1 Library modules (`src/ehmbrain/`)

`perf/cycle.py` — the engine model. Defines four model classes and their builders.

- `CFM56(pyc.Cycle)`: one thermodynamic point. Builds the component graph (inlet, fan, splitter, booster, HPC with three bleed ports, post-compressor bleed, combustor, cooled HPT/LPT, ducts, two convergent nozzles, two shafts, performance block) and its balance system. In *design* mode the four balances solve mass flow for thrust, fuel–air ratio for T_{t4} , and both turbine expansion ratios for zero net shaft power. In *off-design* mode they solve mass flow and bypass ratio to hold the design nozzle throat areas, both spool speeds for shaft power balance, and fuel–air ratio according to the throttle mode: **T4** (rating temperature), **percent_thrust** (a fraction of a reference thrust) or **N1** (hold fan speed, the CFM56 rating parameter, implemented through a passthrough component because the speed is itself another balance’s output). Each point

carries its own Newton solver (tolerance 10^{-8} , Armijo–Goldstein line search, subsystem solves enabled).

- **MPCFM56(pyc.MPCycle)**: DESIGN + the four SLS anchors A2–A5 as thrust-matched `percent_thrust` points. Used for calibration (Section 4.4).
- **MPCFM56Study**: DESIGN + one off-design point whose five turbo-machine map scalars (efficiency and flow) pass through multiplier components (`health_<comp>.eta_fac/flow_fac`); setting a factor to $1 + \Delta$ imposes a health deviation. Used for the ICM and, in F2, for degraded-fleet snapshots.
- **MPCFM56Deck**: DESIGN + `OD_max` (maximum thrust at the rating T_4) + `OD_pt` (a commanded fraction of that maximum). Used for the baseline decks.
- **Builders** (`build_problem`, `build_study_problem`, `build_deck_problem`) return solver-ready problems with all design inputs and balance initial guesses set; `solve_anchors()` encapsulates the idle power continuation ($0.30 \rightarrow 0.07$); `snapshot()` reads the full sensor set at a point; `set_health()` imposes a health-deviation dictionary.

perf/icm.py — **GPA sensitivity analysis.** `compute_icm()` builds a Study problem per health parameter (in parallel worker processes, norm N1), perturbs it $\pm 0.5\%$ at constant N1 and forms the central-difference column in %-per-%; `svd_report()` returns rank, singular values and condition number; `signature_angles()` the pairwise angles between fault signatures; and `confusable_pairs()` the sub- 15° subset that instantiates hypothesis H2.

common/corrections.py — **corrected parameters.** ISA atmosphere (`isa_static`), ram-corrected θ/δ at the fan face (`theta_delta`), the exponent regression `fit_correction_exponents()` (log Y against a cubic polynomial in log corrected speed plus $a \log \theta + b \log \delta$, solved by least squares) and `correct()` to apply the fitted law.

3.5.2 Executable scripts (scripts/)

Each script is a pipeline stage writing versioned artifacts under `data/processed/`:

Script	Function and outputs
<code>run_design_point.py</code>	Solves the design point, checks the five WP1.1 acceptance windows against the contract; nonzero exit on failure.
<code>run_anchors.py</code>	Solves DESIGN + A2–A5 with the idle continuation and compares fuel-flow/OPR predictions to the EEDB (gated tolerances); prints the verdict table.
<code>make_decks.py</code>	Envelope sweep for the baseline decks: independent (altitude, Mach) blocks in parallel processes, warm continuation inside a block, cold-rebuild recovery (norm N2), then a per-channel parallel leave-one-out audit. → <code>baseline_deck.parquet</code> , <code>deck_report.json</code> .
<code>make_corrected_baseline.py</code>	Fits the correction exponents on the deck and re-runs the leave-one-out audit in corrected space. → <code>corrected_baseline.json</code> .
<code>make_icm.py</code>	ICM at the six-point operating grid (columns in parallel), SVD and confusability analysis. → <code>icm_<point>.npz</code> , <code>icm_report.json</code> .
<code>cea_crosscheck.py</code>	Re-solves the design point with CEA thermodynamics and reports deltas against tabular. → <code>cea_crosscheck.json</code> .
<code>make_report_assets.py</code>	Regenerates every figure and table of this document from the model and the artifacts above (norm N4). → <code>paper/report/figures/*.pdf</code> , <code>paper/report/tables/*.tex</code> , <code>assets.json</code> .

The data flow is linear: `conf/cfm56_7b_targets.yaml` (the contract) feeds the model classes; runners and `make_*` scripts solve them and write `data/processed/*` artifacts; `make_report_assets.py` turns artifacts into report evidence; the tests gate every stage.

3.5.3 Test suite (tests/)

Sixteen tests, all run in CI on every change: environment smoke (the vendored pyCycle example end-to-end); design-point convergence and acceptance windows; anchor convergence, fuel-flow and OPR tolerances, and N1/N2 redlines; the Study self-consistency property (healthy off-design at design N1 reproduces the design point); ICM physical signs and rank-3 cockpit underdetermination; and the corrections module (ISA values, θ/δ , exponent-fit recovery on synthetic data with known truth).

3.6 Comparison metrics

Both families are scored with identical metrics. For detection: lead time (how many flights before the ground-truth event the method raises a sustained alert), false alarms per thousand flights, F1 and precision-recall AUC. For isolation: top-1/top-2 accuracy, macro-F1, a *smearing index* (the fraction of

$\|\hat{\mathbf{x}}\|_1$ assigned to healthy components) and the PCS. For health estimation: RMSE of $\hat{\mathbf{x}}$ against ground truth. For RUL: RMSE, MAE, the NASA PHM08 score

$$S = \sum_i s_i, \quad s_i = \begin{cases} e^{-d_i/13} - 1 & d_i < 0 \\ e^{d_i/10} - 1 & d_i \geq 0 \end{cases}, \quad d_i = \widehat{\text{RUL}}_i - \text{RUL}_i, \quad (3.1)$$

which penalizes late predictions more than early ones (a late removal risks an in-flight shutdown; an early one merely wastes some life). Two further prognostic metrics need definitions: α - λ *accuracy* is the fraction of predictions falling within $\pm\alpha$ (here 20 %) of the true RUL when evaluated at specified life fractions λ (here 50, 70 and 90 % of the way to failure) — it asks “was the prediction acceptable at mid-life? near the end?”; the *prognostic horizon* is the earliest moment from which the prediction stays within the $\pm\alpha$ band for the rest of the engine’s life — longer horizons mean earlier trustworthy warnings. For uncertainty: empirical versus nominal interval coverage (does the “90 %” interval actually contain the truth 90 % of the time?) and mean interval width (narrow honest intervals are the goal; wide intervals are trivially calibrated but useless).

Chapter 4

Thermodynamic twin of the CFM56-7B26

This chapter documents the completed part of phase F1: the CFM56-7B26 performance model in pyCycle [5], its design point (work package WP1.1) and its off-design calibration against certified measured data (WP1.2). All result figures and tables are produced by the evidence-generation script (Section 3.4).

4.1 What a cycle model is

A 0-D *cycle model* represents the engine as a chain of thermodynamic components—inlet, fan, compressors, combustor, turbines, nozzles—each transforming the air’s pressure, temperature and composition according to its governing equations, without resolving any internal geometry. Each turbomachine carries a *component map*: a table, obtained from rig tests or scaled from a similar machine, giving its efficiency and flow as a function of rotational speed and operating point. The model runs in two modes:

Design mode sizes the engine: given targets (thrust, component pressure ratios, turbine inlet temperature), it computes the flow areas, map scale factors and shaft-work balance that a machine meeting those targets must have.

Off-design mode answers the service question: given the now-fixed geometry, what does the engine do at any other flight condition and thrust setting? This is the mode the EHM baseline decks and the influence coefficient matrix are built from.

pyCycle solves both modes with a Newton solver: it iterates on a set of unknowns (mass flow, fuel–air ratio, spool speeds, map operating points) until a set of residual equations (nozzle-area match, shaft power balance,

thrust target) is driven to zero. Newton solvers are powerful but fragile far from a good initial guess —a fragility that shaped several decisions below.

4.2 Strategy: adapt, don’t build from scratch

pyCycle ships a validated example of a two-spool, separate-flow, cooled high-bypass turbofan (`high_bypass_turbofan.py`) with the full component graph, the turbine-cooling bleed network, shaft balances and off-design solver wiring already working. Building the CFM56 from a blank file would re-derive all of that plumbing and typically dies in Newton divergence. Instead, a *morphing* protocol is adopted: the example is vendored into the repository as a known-good regression reference, and the model is steered toward the CFM56-7B26 by changing **at most three parameters per step**, requiring convergence at every step before the next. Every converged step is a commit; if a step diverges, the change is bisected.

All design decisions were fixed in writing *before* coding (`docs/f1-model-spec.md`): the -7B26 variant; tabular thermodynamics for speed with a CEA cross-check planned (CEA is NASA’s chemical-equilibrium code —more accurate, roughly an order of magnitude slower); the design point at cruise, Mach 0.78 / 35 000 ft / ISA (the “aerodynamic design point” convention: an engine is designed for cruise, where it spends most of its life, and takeoff is an off-design condition); pyCycle’s generic component maps, scaled —no public CFM56 maps exist, a declared limitation; thrust set through corrected N1 in off-design (the -7B is an N1-rated engine); pyCycle-native units inside the model with SI only at the boundary; and a table of anticipated solver failure modes, each with a planned countermeasure —one of which proved prophetic (Section 4.4.3).

4.3 Design point (WP1.1)

The cycle graph comprises: flight conditions, inlet, fan, flow splitter, booster, HPC with three bleed ports (HPT vane and blade cooling, customer bleed), a post-compressor bleed feeding the LPT, combustor, cooled HPT and LPT, ducts with pressure losses, convergent core and bypass nozzles, the two shafts, and an accessory power extraction of 250 hp on the HP shaft (driving the aircraft’s generators and pumps). In design mode, four implicit balances close the cycle: the inlet mass flow W meets the thrust target; the fuel–air ratio (FAR) meets the target turbine inlet temperature T_{t4} ; and the HPT and LPT expansion ratios zero the net power on each shaft (each turbine produces exactly what its compressors and losses consume).

The design point converges with a Newton residual below 10^{-6} and satisfies all five acceptance windows defined a priori: BPR 5.10 ± 0.2 (the EEDB target), OPR within its window, T_{t4} within a plausible band, total

Table 4.1: Design-point results (Mach 0.78, 35 000 ft, ISA). Auto-generated from the model.

Quantity	Value at the design point
Net thrust F_n	24.38 kN (5480 lbf)
TSFC	0.6214 lb/(lbf·h)
Bypass ratio (BPR)	5.10
Overall pressure ratio (OPR)	30.03
T_{t4}	1587 K (2857 °R)
Inlet mass flow $\dot{W}$	142.0 kg s ⁻¹
Fuel-air ratio (FAR)	0.0251
HPT expansion ratio	3.586
LPT expansion ratio	4.288
Total cooling/bleed fraction	0.239

Table 4.2: Total conditions at each gas-path station at the design point. Auto-generated.

Station	Location	P_t [bar]	T_t [K]
2	Fan inlet	0.358	245.5
21	Fan exit	0.603	289.5
25	HPC inlet	1.149	354.2
3	HPC exit	10.739	703.9
4	Burner exit	10.159	1587.2
45	HPT exit	2.833	1141.6
5	LPT exit	0.657	806.3

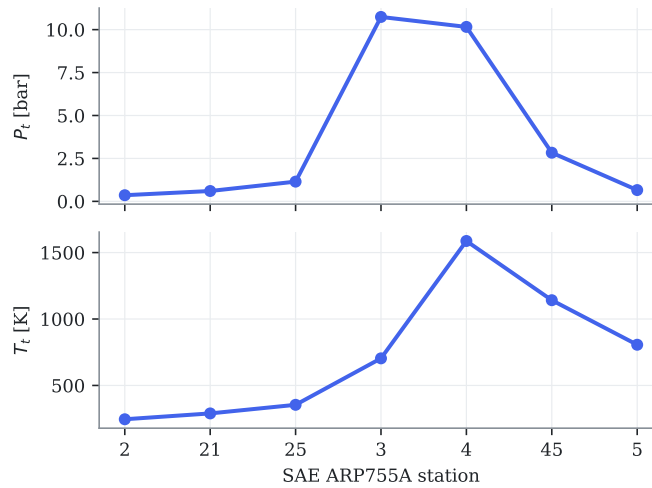


Figure 4.1: Total pressure and temperature along the gas path at the design point. Compression peaks at station 3 (~ 11 bar, Table 4.2), heat addition at station 4, and work extraction across stations 45 and 5.

cooling-plus-bleed fraction between 18 and 25 % of core flow, and thrust exactly on target (Table 4.1). The resulting TSFC ($\approx 0.62 \text{ lb}/(\text{lb}\cdot\text{h})$) lands within 2 % of the publicly quoted cruise value for this engine *without having been used as a calibration target* —an encouraging consistency check.

4.4 Calibration against measured data (WP1.2)

4.4.1 Data sources and philosophy

Off-design calibration uses exclusively **measured, certified data** (Table 2.1). The TCDS provides limits and rated thrusts. The ICAO Engine Emissions Databank provides something less well known: sea-level test-bed *measured fuel flows* at the four landing-takeoff-cycle thrust settings (100, 85, 30 and 7 % of rated thrust F_{00}), plus the engine pressure ratio at rated output (27.61) —all for the exact -7B26 variant, traceable to certification testing, and free. Most academic engine models ignore this source.

The anchor points are solved *thrust-matched*: the fuel–air ratio balance is set to hold $F_n = \text{PC} \cdot F_{00}$ (PC being the power fraction). The model’s fuel flow at each anchor is therefore a genuine **prediction** confronted with the EEDB measurement —not a quantity fitted to it.

4.4.2 Calibration iterations

The first run (model freshly adapted from the example, not yet calibrated to the SLS data) already predicted takeoff fuel flow within -2.1% , but showed two quantitative defects:

1. **N2 above its redline at rated thrust**: 15 311 rpm against the certified limit of 15 183 rpm. Cause: the reference speed used to scale the generic HPC map —the map’s speed-versus-flow shape is not the real compressor’s, so the speed labels it implies must be anchored deliberately. Fix: HP-shaft design reference from 14 324 to 13 940 rpm, placing rated-thrust N2 at 14 915 rpm ($\approx 103 \%$), inside the limit.
2. **Takeoff pressure ratio 28.99 versus the measured 27.61** ($+5.0 \%$). Fix: HPC design pressure ratio from 9.81 to 9.35. The measured value takes precedence over the initial derived estimate, and the cruise OPR becomes a consequence (≈ 30).

After both corrections (Table 4.3 and Figs. 4.2 and 4.3), the three gated anchors meet their tolerances: takeoff at -1.40% fuel-flow error with the pressure ratio within $+0.3 \%$ of the measured value and both shaft speeds inside their redlines; climb-out at -2.23% ; approach at -3.08% .

Table 4.3: SLS anchors versus the ICAO databank measurements after calibration. W_f model is the model’s prediction at imposed thrust. Auto-generated.

Anchor	$\%F_{00}$	F_n [kN]	W_f model	W_f EEDB	error [%]	tol. [%]	gated
A2 Takeoff	100	117.0	1.204	1.221	-1.40	3.0	✓
A3 Climbout	85	99.4	0.977	0.999	-2.23	3.0	✓
A4 Approach	30	35.1	0.328	0.338	-3.08	5.0	✓
A5 Idle	7	8.2	0.125	0.113	+10.57	5.0	—

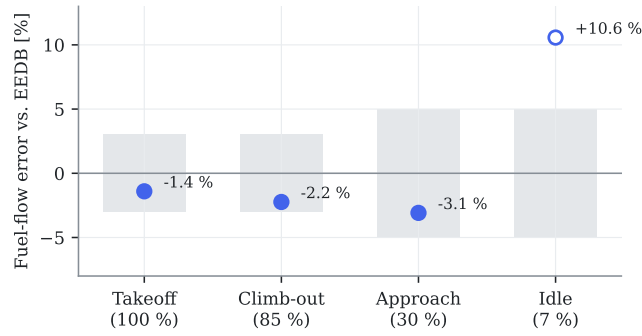


Figure 4.2: Fuel-flow prediction error at each anchor against the EEDB measurement. Gray bands are the contract tolerances ($\pm 3\%$ at takeoff and climb-out, $\pm 5\%$ at approach and idle); the hollow marker (idle) converges but sits outside tolerance and is reported without blocking the milestone (Section 4.4.3).

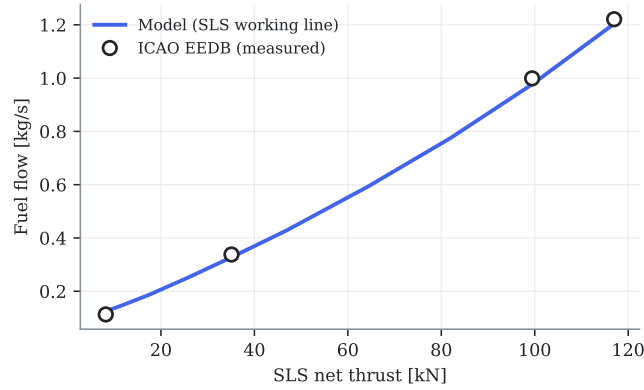


Figure 4.3: The model’s sea-level-static working line (a continuous sweep of thrust setting) against the four measured ICAO databank points. The curvature at low power reflects the degraded accuracy of generic maps far from their scaling point.

4.4.3 Idle convergence: diagnosis and cure

The idle point (7 % of F_{00} , static) systematically diverges from a cold start. At such low power the nozzle pressure ratios approach unity —the exhaust barely pushes— and the nozzles’ internal static-pressure sub-solver bounces against its lower bound, leaving the global Newton iteration without a descent direction. Worse, the failed attempt leaves the point’s internal states (flow stations, map positions) corrupted, so re-seeding only the balance variables does not rescue it: the idle point must *never* start cold at its final setting. The cure —anticipated as a generic countermeasure in the specification— is **power continuation**: the point starts at 30 % of F_{00} (where it converges from cold, just like the approach anchor) and walks down warm through $0.30 \rightarrow 0.20 \rightarrow 0.14 \rightarrow 0.10 \rightarrow 0.08 \rightarrow 0.07$, re-solving at each step. The logic is encapsulated in the library (`solve_anchors()`) and covered by the tests.

With continuation, idle converges and predicts fuel flow with +10.6 % error. This is deep map-extrapolation territory —the generic map’s speed-flow shape at very low power is not the real compressor’s— a known limitation of the approach; the anchor is **reported but not gated** for milestone H1, a decision recorded in the calibration contract (`gated: false`).

4.5 Baseline decks (WP1.3)

Both EHM pipelines need a *baseline*: what a healthy engine reads at any operating condition, so that measured values can be turned into deviations. The baseline is produced by sweeping the healthy model over a realistic flight envelope —altitude blocks with their plausible Mach ranges (no Mach 0.8 at sea level), ambient temperature offsets of 0, +15 and +30 °C above ISA, and six thrust settings from 50 to 100 % of the locally available maximum. The 50 % floor is deliberate: airline EHM trending consumes takeoff, climb and stable-cruise reports, all high-power conditions; approach and idle are neither trending conditions nor reliable model territory (Section 4.4.3). At each condition the model first finds the maximum thrust available at the rating temperature (point `OD_max`, throttled by T_4), then solves the part-power point at the commanded fraction (`OD_pt`) —the same two-point pattern used by the pyCycle reference example. The rating temperature is fixed at $T_{4,\max} = 3200^\circ\text{R}$, a rounded envelope constant chosen from the calibrated takeoff anchor (imposing the certified rated thrust at SLS required $T_4 \approx 3175^\circ\text{R}$, Table 4.3); a real FADEC modulates its rating limits across the envelope, so this constant is a declared simplification —adequate here because the baseline uses *fractions* of the local maximum, not its absolute value.

Two engineering lessons from WP1.2 shaped the sweep. First, the walk is ordered for warm starting within each block (thrust descending), and the independent (altitude, Mach) blocks run in *parallel processes*, one Problem per

worker (project norm N1: long decomposable computations are parallelized; the sweep takes 3.5 minutes on all cores versus 10 minutes serial). Second—the lesson learned the hard way—a failed Newton attempt corrupts the point’s internal states beyond re-seeding, so the recovery action on any failure is to *rebuild* the problem cold with altitude-appropriate guesses (norm N2); a first version of the sweep without this recovery lost every point downstream of the first divergence.

The result is stored as a table plus a scattered linear interpolator (Delaunay triangulation with barycentric interpolation, SciPy’s `LinearNDInterpolator`) in the four operating coordinates, and its quality is measured by leave-one-out cross-validation: each grid point is removed, predicted from all the others, and the relative error recorded — Table 4.5 reports the per-channel median, 95th percentile and maximum. Median errors are a few tenths of a percent; the tail percentiles are dominated by envelope-corner points whose interpolation simplices span across altitude blocks in raw physical coordinates.

4.5.1 Corrected-parameter exponents and the corrected-space baseline

The fix for those tails is the one anticipated in Section 2.3: build the baseline in *corrected-parameter* space. The correction exponents are fitted on the deck itself rather than copied from generic tables: per channel Y , a linear least-squares fit of

$$\log Y = \underbrace{\sum_{k=0}^3 c_k (\log N1_c)^k}_{\text{power line}} + a \log \theta + b \log \delta, \quad (4.1)$$

where the cubic polynomial absorbs the dependence on the corrected speed (the “power line”) and the exponents (a, b) capture whatever ambient dependence remains — exactly the quantities the correction $Y_{\text{corr}} = Y/(\theta^a \delta^b)$ then removes. The fitted exponents land squarely on their textbook values without having been told them (Table 4.4): fuel flow $a = 0.63$, $b = 1.02$ (handbooks quote $\theta^{0.5-0.7} \delta$); N2 $a = 0.49$, $b \approx 0$ (the $\sqrt{\theta}$ corrected-speed law); EGT $a = 0.95$, $b \approx 0$ (temperature ratio). This is a further independent consistency check of the model physics.

Re-running the leave-one-out audit on the corrected channels over (corrected N1, Mach) collapses the tails by an order of magnitude (Table 4.4): worst-case N2 error drops from 9.7 % to 0.5 %, EGT from 24 % to 2.0 %, fuel flow from 91 % to 6.6 %. Two remarks for honesty: net thrust is excluded—it is not an EHM sensor channel (it comes from the thrust-setting definition), and it does not collapse under a θ/δ law at fixed N1c because of its strong direct Mach dependence; and the residual corrected-space errors are comparable to the sensor noise floor that phase F2 will inject ($\sigma_{\text{EGT}} \approx 0.3$ %,

$\sigma_{W_f} \approx 0.5\%$). Baseline error is also common-mode: both EHM pipelines consume the same baseline, so it does not bias the comparison.

Table 4.4: Fitted correction exponents per channel and the leave-one-out error before (raw 4-D coordinates) versus after (corrected space). Auto-generated.

Channel	a (θ)	b (δ)	raw LOO P95/max [%]	corrected LOO P95/max [%]
Fuel flow	0.628	1.022	15.77 / 90.6	3.08 / 6.6
N2	0.495	0.007	2.65 / 9.7	0.34 / 0.5
EGT	0.949	0.002	6.01 / 24.1	1.12 / 2.0

Table 4.5: Leave-one-out interpolation error of the baseline deck, per channel. Auto-generated.

Channel	Median [%]	P95 [%]	Max [%]
Fuel flow	0.498	15.767	90.603
N1	0.268	6.369	25.721
N2	0.080	2.649	9.697
EGT	0.167	6.012	24.062
Net thrust	0.000	16.437	85.402

4.6 Influence coefficient matrix and observability (WP1.4)

4.6.1 Health-parameter injection

To compute the ICM —and, in phase F2, to simulate degraded engines—the model must accept imposed health deviations. `pyCycle` scales its generic component maps with per-component design scalars (efficiency and flow); the off-design points normally inherit them from the design point. In the `MPCFM56Study` model those connections are routed through multiplier components, so that any efficiency or flow-capacity deviation can be dialed in per run. Two implementation subtleties are worth recording: (i) the multipliers must execute *before* the off-design point —the multi-point group runs its children in insertion order, and a mis-ordered version silently solved the off-design point with unscaled maps; the symptom (a “healthy” point that did not reproduce the design point) was caught by the self-consistency check below; and (ii) holding N1 constant, the CFM56’s rating parameter, requires a fuel-flow balance targeting the LP spool speed through a passthrough component, since the speed is itself the output of another balance.

The wiring is validated by a self-consistency property that is now a permanent test: the *healthy* off-design point at cruise with N1 equal to

its design value must reproduce the design point exactly. It does: thrust 5480.0 lbf, N2 13 940.0 rpm, OPR 30.03, EGT 1141.6 K —machine-precision agreement with Table 4.1.

4.6.2 The ICM at the reference conditions

The ICM is computed by central finite differences ($\pm 0.5\%$ per health parameter) at the two conditions where airline EHM snapshots are taken: cruise (M0.78/35 000 ft) and sea-level takeoff, at constant N1. Figure 4.4 shows the cruise matrix for the extended sensor set; every sign satisfies the physical expectations (e.g. a healthier HPT runs cooler and thriftier: negative EGT and W_f entries in its efficiency column), and these sign checks run automatically in the test suite.



Figure 4.4: Influence coefficient matrix at cruise (extended sensor set): % change of each measured channel per +1 % change of each health parameter. Auto-generated from the model.

4.6.3 Observability: the quantitative case for hypothesis H2

With the cockpit set only (N2, W_f , EGT —N1 is the held rating parameter, so it carries no information), the ICM has rank 3 for 10 unknowns and the geometry of its columns is damning: the minimum angle between fault signatures is **1.3 degrees**, with seven pairs below 15 degrees (Table 4.7). The most confusable pair, HPC efficiency versus HPT efficiency, is precisely the classical HPC/HPT ambiguity reported in the GPA literature [9] —here quantified for this engine. Two faults whose signatures differ by 1.3 degrees are, at realistic noise levels, indistinguishable to any linear estimator: this is the smearing mechanism measured, and it defines the “confusable” subset on which hypothesis H2 will compare the AI’s isolation ability against WLS-GPA. The extended sensor set (adding P25/T25/PS3/T3) raises the rank to

7 and multiplies the minimum angle by roughly five —the quantitative basis for the sensor-set ablation of contribution C6.

The analysis is repeated over a six-point grid spanning the envelope —both snapshot conditions plus power, altitude, climb and hot-day variations (Table 4.6). The observability structure barely moves: the minimum cockpit signature angle stays between 1.2 and 1.4 degrees and the confusable-pair count is constant at seven. The smearing problem is therefore *intrinsic to the cockpit sensor set*, not an artifact of one flight condition —which both justifies interpolating the ICM over this grid in phase F3 and forecloses the easy escape of “diagnose somewhere else in the envelope”.

Table 4.6: Observability of the cockpit ICM across the operating-point grid. Auto-generated.

Operating point	cond. (cockpit)	min angle ckpt. [deg]	confusable pairs	min angle ext. [deg]
Cruise M0.78/FL350	12.2	1.32	7	8.62
Cruise, low power	13.0	1.20	7	6.68
Cruise FL390	11.3	1.37	7	8.60
Climb 10 kft/M0.4	12.6	1.35	7	7.38
Takeoff SLS	10.5	1.33	7	7.92
Takeoff SLS, ISA+30	11.4	1.25	7	8.97

Table 4.7: Confusable fault pairs at cruise with cockpit sensors (signature angle $< 15^\circ$). Auto-generated.

Fault A	Fault B	Angle [deg]
η_{HPC}	η_{HPT}	1.32
η_{FAN}	η_{LPT}	4.32
η_{HPT}	Γ_{HPT}	6.02
Γ_{FAN}	η_{LPT}	6.52
η_{HPC}	Γ_{HPT}	7.20
η_{FAN}	Γ_{FAN}	10.05
η_{FAN}	η_{LPC}	13.53

4.6.4 Thermodynamics cross-check

Decision D2 (tabular thermodynamics for speed) is closed by re-solving the design point with NASA’s chemical-equilibrium code (CEA): all key quantities agree within 0.7 % (Table 4.8), an order of magnitude below the ± 3 % calibration tolerance. Tabular thermodynamics is therefore retained for all fleet-scale computation.

Table 4.8: Design point solved with tabular versus CEA thermodynamics. Auto-generated.

Quantity	Tabular	CEA	Δ [%]
Net thrust [lbf]	5480.0000	5480.0000	-0.000
TSFC [lb/(lbf·h)]	0.6214	0.6249	+0.553
Inlet mass flow [lbm/s]	313.0851	315.2029	+0.676
Fuel-air ratio	0.0251	0.0250	-0.122
HPT expansion ratio	3.5865	3.5902	+0.104
LPT expansion ratio	4.2879	4.3109	+0.537

4.7 Declared limitations

The model is *representative*, not certifiable, and its limits are declared for citation in the results chapters: (i) no transients —steady-state points only; (ii) variable geometry (bleed valves, variable stators) implicit in the maps, not modeled; (iii) no Reynolds or humidity corrections; (iv) absolute N1/N2 labels approximate by construction (generic map shapes), though respecting the certified redlines at the rated point; (v) the model’s EGT is the HPT-exit temperature (station 45), a consistent proxy —identical for ground truth and for both EHM pipelines— of the certified T49.5, whose displayed value additionally carries the shunt and trim functions described in Section 2.2; and (vi) the idle error quoted above. None of these limitations affects the validity of the AI-versus-traditional comparison, which rests on the model’s sensitivity structure (the ICM), not on its absolute values.

4.8 Automated verification

The chapter’s claims are pinned by tests: design-point convergence and acceptance windows; convergence of all four anchors including the idle continuation; fuel-flow and pressure-ratio tolerances at the gated anchors; N1/N2 redlines at rated thrust; the Study self-consistency property (healthy off-design = design); the ICM physical sign checks and its rank-3 cockpit underdetermination; and the pyCycle environment smoke test. Twelve tests green in continuous integration at the time of writing.

Chapter 5

Work in progress and planning

5.1 Phase F1: closed

All four work packages of phase F1 are complete and documented in Chapter 4: calibrated design point (WP1.1), measured-data anchor calibration (WP1.2), baseline decks with fitted correction exponents and a corrected-space interpolator (WP1.3), and the ICM over a six-point operating grid with its observability analysis and the CEA cross-check (WP1.4). Milestone **H1 is declared closed** with one documented deviation from the original gate wording: the leave-one-out target of 0.1 % was set before the baseline space was chosen; the achieved corrected-space errors (medians 0.1–1 %, Table 4.4) sit at the sensor-noise floor injected in phase F2 and are common-mode to both EHM pipelines, so the gate intent —a baseline whose error does not distort the comparison— is met. Optional refinement (grid densification near block boundaries) is queued as non-blocking.

5.2 Remaining phases

5.3 Milestone log

Table 5.1: Remaining phases, key deliverables and closing criteria.

Phase	Key deliverable	Closing criterion (gate)
F2	SynCFM56 v1.0: synthetic fleet of ~ 100 engines with ground truth and ACARS-style snapshots	H2: realism and difficulty audit passed; no leakage between splits; dataset datasheet written
F3	Complete traditional pipeline (baseline, trending, WLS, Kalman, rules, extrapolation RUL)	H3: full metrics on test; smearing measured against ground truth
F4	AI suite (detection, diagnosis, RUL, physics-informed hybrid, conformal intervals, PCS)	H4: beats the traditional pipeline on ≥ 1 primary metric per task on validation
F5	Pre-registered evaluation + ablations + N-CMAPSS	H5: hypotheses H1–H5 resolved with statistical tests; figures reproducible with one command
F6	Case studies, dashboard, final report	H6: full reproduction from raw data; defensible document

Table 5.2: Milestones reached (updated at every gate).

Milestone	Date	Evidence
H0	2026-07-03	Reproducible environment (uv, Python 3.11, pyCycle 4.4 with <code>numpy<2</code>); a complete example cycle converging locally and in CI; repository skeleton and smoke test.
H1 (partial)	2026-07-03	WP1.1 and WP1.2 complete: design point inside all five acceptance windows (Table 4.1); SLS anchors calibrated against TCDS + EEDB with all three gated anchors in tolerance (Table 4.3); calibration contract with verified values.
H1 (WP1.3–1.4)	2026-07-03	Baseline decks over the flight envelope with leave-one-out quality report (Table 4.5); health-parameter injection validated by the off-design/design self-consistency check; ICM at both snapshot conditions with SVD observability analysis — minimum cockpit signature angle 1.3° , seven confusable pairs (Table 4.7); tabular-vs-CEA cross-check within 0.7% (Table 4.8); twelve automated tests green.
H1 closed	2026-07-03	Correction exponents fitted from the model land on textbook values (Table 4.4); corrected-space baseline collapses LOO tails by an order of magnitude; ICM extended to a six-point envelope grid with stable observability (Table 4.6); computations parallelized per norm N1 (decks $2.9\times$, ICM $6\times$ faster); sixteen automated tests green.

References

- [1] Louis A. Urban. “Parameter Selection for Multiple Fault Diagnostics of Gas Turbine Engines”. In: *Journal of Engineering for Power* 97.2 (1975), pp. 225–230. DOI: 10.1115/1.3445969.
- [2] Allan J. Volponi. “Gas Turbine Engine Health Management: Past, Present, and Future Trends”. In: *Journal of Engineering for Gas Turbines and Power* 136.5 (2014), p. 051201. DOI: 10.1115/1.4026126.
- [3] Abhinav Saxena et al. “Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation”. In: *2008 International Conference on Prognostics and Health Management*. 2008, pp. 1–9. DOI: 10.1109/PHM.2008.4711414.
- [4] Manuel Arias Chao et al. “Aircraft Engine Run-to-Failure Dataset under Real Flight Conditions for Prognostics and Diagnostics”. In: *Data* 6.1 (2021), p. 5. DOI: 10.3390/data6010005.
- [5] Eric S. Hendricks and Justin S. Gray. “pyCycle: A Tool for Efficient Optimization of Gas Turbine Engine Cycles”. In: *Aerospace* 6.8 (2019), p. 87. DOI: 10.3390/aerospace6080087.
- [6] European Union Aviation Safety Agency. *Type-Certificate Data Sheet No. E.004 for CFM56-7B Series Engines, Issue 07*. Tech. rep. <https://www.easa.europa.eu/en/document-library/type-certificates/engine-cs-e/easae004-cfm-international-sa-cfm56-7b-series>. EASA, Jan. 2023.
- [7] International Civil Aviation Organization. *ICAO Aircraft Engine Emissions Databank, edition 03/2026*. Hosted by EASA. <https://www.easa.europa.eu/en/domains/environment/icao-aircraft-engine-emissions-databank>. 2026.
- [8] SAE International. *ARP755A: Gas Turbine Engine Performance Station Identification and Nomenclature*. Tech. rep. SAE, 1974.
- [9] Y. G. Li. “Performance-analysis-based gas turbine diagnostics: A review”. In: *Proceedings of the Institution of Mechanical Engineers, Part A: Journal of Power and Energy* 216.5 (2002), pp. 363–377. DOI: 10.1243/095765002320877856.

- [10] Ihor S. Diakunchak. “Performance Deterioration in Industrial Gas Turbines”. In: *ASME 1991 International Gas Turbine and Aeroengine Congress and Exposition*. Journal of Engineering for Gas Turbines and Power, 114(2):161–168. 1992. DOI: 10.1115/1.2906565.
- [11] Rainer Kurz and Klaus Brun. “Fouling Mechanisms in Axial Compressors”. In: *Journal of Engineering for Gas Turbines and Power* 134.3 (2012), p. 032401. DOI: 10.1115/1.4004403.
- [12] Justin S. Gray et al. “OpenMDAO: An open-source framework for multidisciplinary design, analysis, and optimization”. In: *Structural and Multidisciplinary Optimization* 59 (2019), pp. 1075–1104. DOI: 10.1007/s00158-019-02211-z.